

DOCUMENT 3: COMPLETE GUIDE TO EVERY MACHINE LEARNING MODEL USED IN THIS PROJECT

Simple English Guide: HOW, WHY, and WHEN for All 4 Regression Algorithms

Introduction

This document provides an in-depth, plain-English breakdown of all **4 Machine Learning Regression Algorithms** developed, tuned, and benchmarked in our **Loan Amount Prediction System**:

1. **Linear Regression** (Foundational Baseline)
2. **Decision Tree Regression** (Rule-Based Partitioning)
3. **Random Forest Regression** (Bagged Ensemble)
4. **Gradient Boosting Regression** (Sequential Error-Correcting Champion)

For every model, this document explains:

1. **WHAT it is:** Plain-English intuition and real-world analogy.
 2. **HOW it works:** Mathematical principles explained simply.
 3. **WHY we chose it:** Its purpose in our loan underwriting project.
 4. **WHEN to use it (and when NOT to):** Production guidance.
 5. **Exact Performance in our Project:** MAE, MSE, RMSE, R^2 , and ranking analysis.
-

Model 1: Linear Regression (The Foundational Baseline)

1. WHAT is Linear Regression?

Linear Regression is the grandfather of predictive modeling. Imagine drawing a single straight line through a scatter plot of dots so that the line sits as close to every dot as possible. It assumes that as input numbers go up or down, the output moves in a constant, straight-line proportion.

2. HOW does it work?

- **Mathematical Equation:**

$$\hat{y} = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p$$

Where:

- $\hat{y}$ is the predicted loan amount.
- β_0 is the intercept (base loan amount).
- $X_1, X_2, \dots$ are the applicant features (income, assets, requested loan).
- $\beta_1, \beta_2, \dots$ are the learned weights (slopes) showing how much the loan increases for each unit change in that feature.
- **Optimization (Ordinary Least Squares):** The algorithm calculates the mathematical derivative to find the exact weights that minimize the sum of squared errors:

$$\text{Loss} = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

3. WHY we chose it:

It provides our **foundational benchmark**. Before testing complex algorithms, every data scientist must test Linear Regression to establish a performance baseline. It is also 100% transparent and interpretable.

4. WHEN to use it:

- **USE WHEN:** The relationship between features and target is mostly linear, interpretability is required by law, and you need lightning-fast inference.
- **DO NOT USE WHEN:** The data has sharp threshold cutoffs (e.g., credit score cliff-effects) or non-linear compounding interactions.

5. Performance in our Loan Project:

- **MAE:** ■3,04,394.08
- **MSE:** 2.08×10^{11}
- **RMSE:** ■4,56,024.85
- **Test R^2 Score:** 0.9847 (Train R^2 : 0.9864)
- **Rank:** #4 out of 4
- **Analysis:** Linear Regression performed with high accuracy because requested loan amounts and collateral values correlate linearly with sanctioned limits, but it lacks the capacity to model subtle risk haircuts.

Model 2: Decision Tree Regression (Rule-Based Partitioning)

1. WHAT is a Decision Tree?

A Decision Tree is like playing a game of "20 Questions." It makes predictions by asking a sequence of Yes/No questions about the applicant (e.g., "Is Credit Score > 750? If Yes, is Collateral Asset Value > ■50 Lakhs?").

2. HOW does it work?

- **Recursive Splitting:** At every step (node), the algorithm searches through every feature and every possible numerical cutoff to find the split that maximizes the reduction in Mean Squared Error (variance reduction):

$$\Delta \text{MSE} = \text{MSE}_{\text{parent}} - \left(\frac{N_{\text{left}}}{N} \text{MSE}_{\text{left}} + \frac{N_{\text{right}}}{N} \text{MSE}_{\text{right}} \right)$$

- **Leaf Predictions:** Once an applicant reaches a terminal leaf node, the predicted loan amount is simply the average loan amount of all training applicants who landed in that same bucket.
- **Pruning & Constraints:** To prevent the tree from growing until each leaf contains only 1 applicant (pure memorization), we constrained it with `max_depth=9` and `min_samples_leaf=10`.

3. WHY we chose it:

Banking underwriting policies operate using strict rule-based tiers (e.g., "Applicants with credit score below 600 receive an automatic sanction haircut"). Decision trees mirror how human loan officers think.

4. WHEN to use it:

- **USE WHEN:** You need human-readable decision rules and the data has categorical step changes.
- **DO NOT USE WHEN:** You need smooth numeric predictions. Single decision trees produce jagged "staircase" predictions and have higher variance.

5. Performance in our Loan Project:

- **MAE:** ■2,74,960.44
- **MSE:** 1.89×10^{11}
- **RMSE:** ■4,34,535.01
- **Test R^2 Score:** 0.9861 (Train R^2 : 0.9900)
- **Rank:** #3 out of 4
- **Analysis:** Outperformed Linear Regression (reducing RMSE by over ■21,000) by successfully segmenting credit score and default thresholds.

Model 3: Random Forest Regression (The Bagged Ensemble)

1. WHAT is a Random Forest?

"Wisdom of the crowd." Instead of relying on a single decision tree that might make mistakes, Random Forest grows **100 independent decision trees** and averages their predictions together.

2. HOW does it work?

- **Bootstrap Aggregation (Bagging):** Each tree is trained on a random sample of the data chosen with replacement (some rows are duplicated, some left out).
- **Feature Subsampling:** At every split, each tree only gets to consider a random subset of features. This prevents all 100 trees from looking identical.
- **Averaging:**

$$\hat{y} = \frac{1}{B} \sum_{b=1}^B T_b(x)$$

The individual errors of individual trees cancel each other out, producing a smooth, robust prediction.

3. WHY we chose it:

Random Forest is one of the most reliable "off-the-shelf" algorithms in machine learning. It almost never overfits severely and handles non-linearities and outliers with ease.

4. WHEN to use it:

- **USE WHEN:** You want high accuracy with minimal tuning, and your dataset contains both numerical and categorical features with complex relationships.
- **DO NOT USE WHEN:** You need tiny model file sizes (a forest of 100 deep trees can take 10+ megabytes).

5. Performance in our Loan Project:

- **MAE:** ■2,58,950.02
- **MSE:** 1.60×10^{11}
- **RMSE:** ■3,99,623.06
- **Test R^2 Score:** 0.9883 (Train R^2 : 0.9938)
- **Rank:** #2 out of 4
- **Analysis:** Dramatically reduced error compared to a single Decision Tree (RMSE dropped from ■4.34L to ■3.99L), demonstrating the power of ensemble averaging.

Model 4: Gradient Boosting Regression (THE CHAMPION MODEL)

1. WHAT is Gradient Boosting?

If Random Forest is a committee voting simultaneously, Gradient Boosting is a master apprentice system. It builds trees **one after another in sequence**. Each new tree focuses specifically on the mistakes (residuals) made by the previous trees!

2. HOW does it work?

1. Initial Baseline: The model starts by predicting the average loan amount for everyone: $F_0(x) = \bar{y}$.

2. Compute Residual Errors: For every applicant, calculate the error:

$$r_{[i, m]} = y_i - F_{[m-1]}(x_i)$$

3. Fit a New Shallow Tree: A small tree ($max_depth=5$) is trained to predict these residual errors.

4. Update with Shrinkage (Learning Rate η):

$$F_m(x) = F_{[m-1]}(x) + \eta \cdot T_m(x) \quad (\eta = 0.08)$$

Multiplying by a small learning rate ensures the model takes small, careful steps and avoids overshooting the target.

5. Repeat 150 Times: Over 150 rounds, the residual errors shrink closer and closer to zero.

3. WHY we chose it:

Gradient Boosting is the undisputed gold standard for tabular data in machine learning competitions (Kaggle) and enterprise credit scoring. It captures intricate non-linearities better than any other classical algorithm.

4. WHEN to use it:

- **USE WHEN:** Maximizing predictive accuracy on tabular data is your top priority.
- **DO NOT USE WHEN:** Training time is severely limited or data is extremely noisy with severe mislabeled outliers.

5. Performance in our Loan Project:

- **MAE:** █2,50,815.84 (Lowest error across all models!)
- **MSE:** 1.53×10^{11}
- **RMSE:** █3,91,191.33 (Lowest RMSE across all models!)
- **Test R^2 Score:** 0.9888 (Highest R^2 score!)
- **Train R^2 :** 0.9948
- **Rank:** #1 UNDISPUTED CHAMPION
- **Analysis:** Gradient Boosting struck the perfect balance between bias and variance, achieving the highest precision on the test set.

Summary: Final Algorithm Comparison Matrix

Rank	Algorithm	MAE (█)	RMSE (█)	Test R^2	Train R^2	Best Use-Case
#1	Gradient Boosting Regression	█2.50 Lakh	█3.91 Lakh	0.9888	0.9948	Production Deployment
#2	Random Forest Regression	█2.58 Lakh	█3.99 Lakh	0.9883	0.9938	High Reliability & Stability
#3	Decision Tree Regression	█2.74 Lakh	█4.34 Lakh	0.9861	0.9900	Transparent Policy Rules
#4	Linear Regression	█3.04 Lakh	█4.56 Lakh	0.9847	0.9864	Fast, Interpretable Baseline